

MobileVISTA: Generative Data Augmentation for Pose Generalization in Mobile Manipulation

Suzannah Wistreich¹, Stephen Tian¹, Isabella Huang², Vitor Campagnolo Guizilini²,
Sergey Zakharov², Katherine Liu², Jiajun Wu¹

¹Stanford University ²Toyota Research Institute

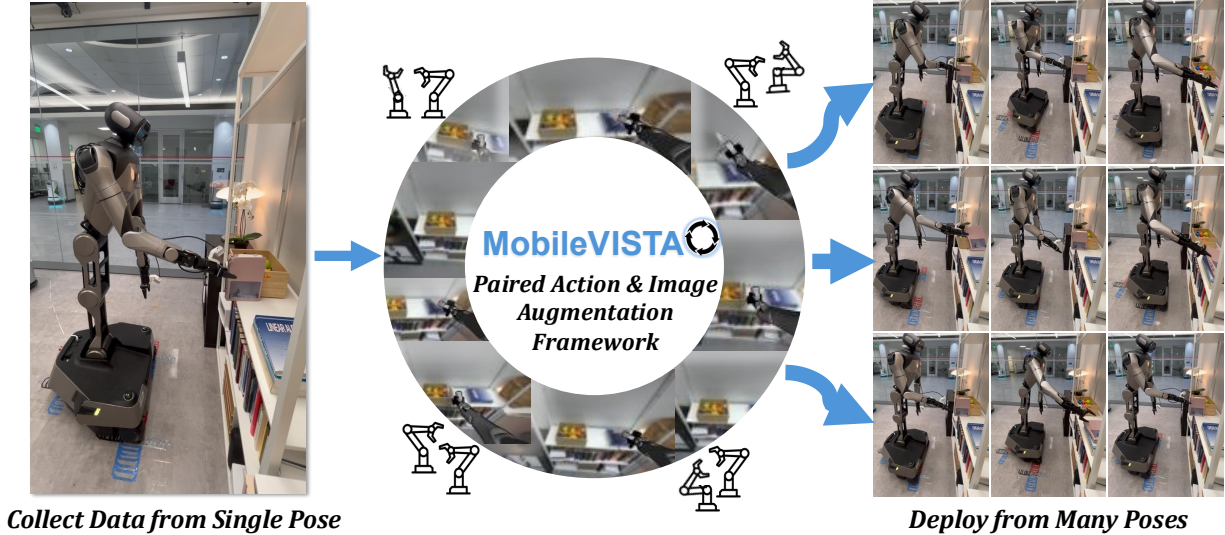


Fig. 1: **MobileVISTA**. Given demonstrations collected at a single canonical robot pose (left), our framework jointly synthesizes egocentric observations and retargeted actions (center) to produce training data that allows policies to succeed from a range of unseen base poses at deployment (right).

Abstract—Mobile manipulators such as humanoid robots are increasingly deployed in dynamic, unstructured environments to perform dexterous manipulation tasks. However, end-to-end manipulation policies trained to imitate demonstration data collected from a single robot pose are brittle: even centimeter-scale deviations in robot pose at deployment can drive egocentric observations and end-effector trajectories out of the training distribution, leading to sharp drops in performance. We introduce MobileVISTA, a data generation framework that transforms demonstrations captured at canonical poses into diverse, pose-perturbed training data by jointly (1) augmenting egocentric visual observations and (2) retargeting actions to compensate for base pose changes. Unlike prior methods, which assume a camera rigidly mounted off the actuated chain or non-trivial articulated robot geometry largely out of frame, MobileVISTA targets compatibility with egocentric platforms (e.g., humanoids) where the camera is both influenced by *and* must observe the robot’s kinematic chain as it moves. We study MobileVISTA in simulated tasks spanning humanoid and bimanual embodiments, and on a real Galaxea R1 Pro. We find policies trained on MobileVISTA-augmented data demonstrate improved robustness to previously out-of-distribution poses encountered at test time, without additional demonstration collection or a trained generative model. Additionally, we find MobileVISTA’s benefit is largest on tested humanoids, where the camera rides the actuated chain and the robot fills much of the frame. Additional videos and appendix can be found on our website: <https://sdwistreich.github.io/mobilevista>

I. INTRODUCTION

Mobile manipulators increasingly execute complex skills in settings where navigation and manipulation are tightly

sequenced, transitioning directly from locomotion to dexterous interaction from whatever configuration state navigation leaves them in. However, visuomotor policies trained to imitate a small set of task-specific demonstrations overfit to the narrow distribution of robot configurations seen during training. Large-scale pretrained policies aim to broaden this distribution [1]–[3], but deploying on a new task often still relies on task-specific demonstrations, with the fine-tuned policy inheriting their restricted pose distribution. These policies therefore fail to generalize beyond the narrow set of base poses and camera viewpoints seen during training. Even centimeter-scale base pose perturbations [4]—such as those arising from reasonable navigation error—can push the robot into out-of-distribution states.

Re-collecting demonstrations from many base poses is often impractical and scales poorly with the diversity of poses met at deployment, so a desirable property is solving tasks from whichever configuration the robot occupies *without* new demonstrations. Several methods synthesize novel visual observations [5]–[7] to emulate camera viewpoint changes with the robot base fixed, leaving action labels unchanged. In *mobile* settings, however, a base shift alters both on-board camera observations, whose viewpoints depend on the full kinematic chain from base to camera, and the actions needed to complete the task. Recent work generates the two jointly [8]–[10], but assumes either a camera fixed rigidly to

the base or a view containing little robot geometry beyond a gripper. Humanoids satisfy neither: the camera rides an actuated torso, so the target viewpoint is settled by the same inverse-kinematics solve that produces the retargeted action, and the arms and torso fill much of that view.

Toward addressing these limitations, we present **MobileVISTA**, a data generation framework that transforms manipulation demonstrations collected from a single canonical base pose into diverse, pose-perturbed training data for mobile manipulation. Our key contributions are:

- MobileVISTA, a data generation framework for pose-robust mobile manipulation that produces physically consistent training data for arbitrary base configurations entirely offline, without additional data collection.
- An explicit geometry-based observation synthesis procedure that removes the robot from the observation, reprojects the scene with the perturbed base pose, and composites a re-rendering of the robot with retargeted joint configurations. One solve yields *both* the extrinsics of the synthesized view and the robot link poses within it, separating our setting from prior work (Table I).
- A controlled comparison against learned novel view synthesis [11]: explicit geometry-based reprojection with robot segmentation and re-compositing better preserves scene geometry and yields stronger downstream policy performance, and benefit scales with how much of the egocentric frame the robot occupies.

We evaluate MobileVISTA in simulation and on a real Galaxea R1 Pro humanoid, improving over baselines that leverage 3D representations and large-scale pretrained policies under unseen base perturbations. **MobileVISTA** offers a practical data generation approach for robust mobile manipulation under deployment conditions where robot pose cannot be precisely controlled.

II. RELATED WORK

Learning robust mobile manipulation. Learned policies for navigation and manipulation have been widely studied for mobile manipulators. End-to-end approaches train whole-body controllers that jointly produce base and arm actions [12]–[14]. In contrast, modular approaches treat navigation and manipulation as sequential, decoupled components [15], [16]. Although this “park-and-manipulate” paradigm is less expressive than end-to-end control, it requires less training data and is more interpretable. A key challenge in modular systems is ensuring reliable manipulation after navigation, as the post-navigation state may fall outside the manipulation policy’s training distribution. ASC [17] addresses this by learning a corrective policy that adapts existing manipulation skills to such states. In contrast, Mobi- π [4], MoTo [18], and N2M [19] optimize the robot’s base pose to remain within the manipulation policy’s training distribution. We instead pursue a complementary direction by designing manipulation policies that are inherently robust to a wide range of base poses.

Synthetic data generation in robot learning. Synthetic data generation is a promising approach to augmenting scarce

Method	Embodiment	Mobile base	Camera on act. chain	Robot vis. past gripper
VISTA [5]	Single-arm	✗	✗	✓
RoVi-Aug [7]	Single-arm	✗	✗	✓
ROPA [6]	Biman.	✗	✗	✓
RoboSplat [27]	Single-arm	✗	✗	✓
EgoDemoGen [8]	Biman.	✓	✗	✓
1001 Demos [10]	Single-arm	✗	✓	✗
R2RGen [9]	Single, biman.	✓	✗	—
MobileVISTA (ours)	Biman., humanoid	✓	✓	✓

TABLE I: **Settings targeted by prior work.** Embodiment lists every platform class each method is evaluated on, in simulation and on hardware. Only two prior methods consider a mobile base, neither carries the camera on the actuated chain, and none is evaluated on a humanoid. R2RGen synthesizes no images (—).

human demonstration data. Some methods generate synthetic images to improve generalization to novel object and scene configurations [20]–[24]. Another application is generating cross-embodiment data, e.g. from human to robot [25] or robot to robot [7]. Synthetic observations can also improve robustness to eye-in-hand observations [26].

Viewpoint-robust demonstration generation. Several recent methods generate paired observation–action data for viewpoint robustness. RoboSplat [27] edits a Gaussian Splatting reconstruction to vary object pose and type, appearance, lighting, and camera pose, while 1001 Demos [10] couples Gaussian-Splatting rendering of an eye-in-hand fisheye camera with trajectory optimization, compositing gripper geometry into rendered views. However, both assume a fixed base and a per-scene reconstruction captured in advance. VISTA [5] and ROPA [6] fine-tune image diffusion models to render the scene and robot from novel third-person poses, while R2RGen [9] avoids image synthesis entirely, requiring that “the RGB-D sensor be fixed during task execution.”

Two distinguishing properties characterize our target setting: (1) the camera rides the actuated chain, so its pose is known only once inverse kinematics resolves a configuration for the retargeted action; and (2) the robot kinematic chain is visible in frame, so synthesis must re-render articulated geometry rather than composite a rigid end-effector. Prior methods satisfy at most one (Table I). Closest to our setting is EgoDemoGen [8], generating paired egocentric observations and actions under planar base perturbations, but whose “egocentric camera is rigidly mounted on the robot head, which is fixed to the base.” MobileVISTA instead targets cameras on the actuated chain, and synthesizes via explicit geometry rather than a per-domain fine-tuned video generator.

We do not run the methods of Table I as baselines. Five methods assume a fixed base and therefore define no action at a perturbed pose. Of the two that consider a mobile base, R2RGen synthesizes no images, and EgoDemoGen, which synthesizes with a learned video generator, has no public implementation as of submission. We instead compare against learned NVS directly, using GEN3C (Section IV-C), and isolate the contribution of robot re-rendering through a *Fixed Robot Geom.* ablation (Section IV-B).

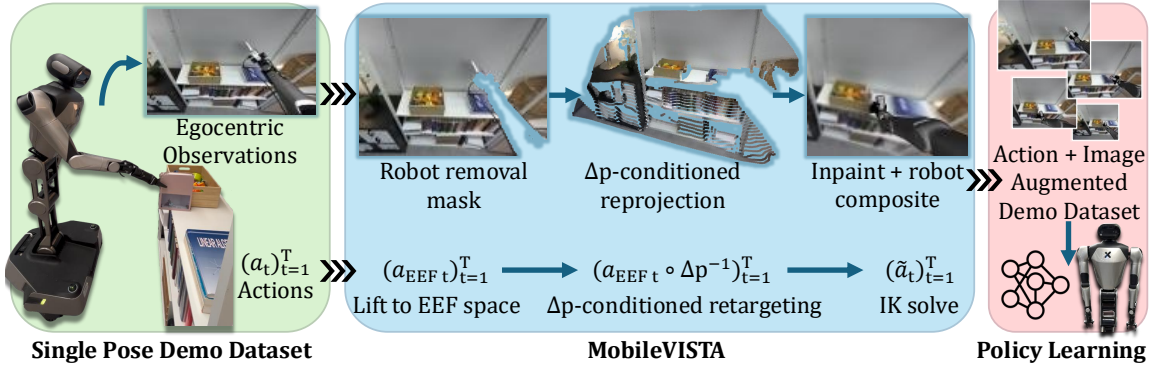


Fig. 2: **The MobileVISTA framework.** Given single-pose demonstrations, MobileVISTA jointly augments observations and actions under a sampled base pose perturbation $\Delta \mathbf{p} \in SE(3)$. The visual branch removes the robot, reprojects the scene, and re-renders the robot at its retargeted configuration, while the action branch retargets actions in end-effector space via $\Delta \mathbf{p}^{-1}$ to produce $(\tilde{a}_t)_{t=1}^T$. This augmented dataset is then used to train a pose-robust policy.

III. THE MOBILEVISTA FRAMEWORK

Given a dataset of expert demonstrations collected from a single canonical robot base pose, MobileVISTA transforms each demonstration into pose-perturbed variants by modifying both egocentric visual observations and their corresponding actions. Training on augmented data exposes policies to diverse robot configurations, enabling them to handle previously out-of-distribution poses at test time. Figure 2 illustrates the method.

Each demonstration consists of time-aligned egocentric and wrist camera RGB-D images, proprioceptive robot states, and joint-space actions including gripper and finger actuation. Joint-space configurations can be mapped to end-effector poses via forward kinematics. Concretely, given a source trajectory $\tau = (o_t, a_t)_{t=1}^T$ collected at canonical base pose $P \in SE(3)$, MobileVISTA synthesizes a pose-perturbed trajectory $\tilde{\tau} = (\tilde{o}_t, \tilde{a}_t)_{t=1}^T$ under randomly sampled pose transformations $P \circ \Delta \mathbf{p}$, where $\tilde{o}_t$ are pose-consistent rendered observations and $\tilde{a}_t$ are retargeted actions aimed to preserve the original end-effector trajectory known to successfully complete a target task. For each synthesized episode, we sample a target base translation offset $\Delta \mathbf{b} \in \mathbb{R}^2$ uniformly within a disk of radius r centered at the canonical pose and a yaw rotation perturbation $\Delta \theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. Together, these define the pose perturbation $\Delta \mathbf{p} \in SE(3)$, the transformation induced by $(\Delta \mathbf{b}, \Delta \theta)$. Given these, MobileVISTA jointly augments (i) visual observations by synthesizing pose-shifted egocentric views of the scene and (ii) action labels by retargeting the original motion to compensate for the base perturbations. We leave wrist views unmodified: their localized viewpoints and the sparse depth of close-range interaction make geometry-based reprojection unreliable, and as retargeting aims to preserve original end-effector trajectories, wrist-camera observations can stay largely consistent under the base perturbation.

A. Generating Visual Observations

To generate pose-perturbed visual observations consistent with a target base offset, we assume access to a simulation model of the robot geometry. The model serves three purposes: (1) segmenting the robot out of the original egocentric

image, (2) re-rendering the robot under retargeted actions to composite with the synthesized scene, and (3) recovering the egocentric camera trajectory those actions induce along the kinematic chain from base to camera.

We adopt a geometry-based segmentation-and-reprojection pipeline to synthesize new egocentric observations. We first render the robot model using the canonical action trajectory and camera parameters to obtain a precise 2D robot geometry mask. Applying this mask to the egocentric depth image removes the robot, yielding a robot-free depth map of the scene. We then de-project the robot-free RGB-D image into a colored point cloud using a pinhole camera model. To account for camera motion induced by retargeting (e.g., torso or waist adjustments), we lift the point cloud into the world frame using the original per-timestep camera simulator extrinsics obtained while rendering the original actions. We apply the target base perturbation in world coordinates, and then re-project the transformed point cloud from the *new* egocentric camera, whose per-timestep extrinsics are obtained by executing the retargeted actions with the robot model. Note that egocentric camera poses are not chosen independently of actions: while a perturbed base pose may allow multiple valid camera poses when intermediate joints are unconstrained, camera extrinsics are inherited from the joint configuration resolved by the retargeted actions (Section III-B) via IK.

This produces a pose-shifted reprojection of the static environment, but introduces missing regions due to both the removal of the original robot *and* newly exposed areas from the scene reprojection. As in prior work [25], we fill these holes, along with any regions missing due to incomplete depth, using the image inpainting model E²FGVI [28]. We inpaint rather than nearest-pixel filling, since disocclusions expose unobserved content near boundaries rather than repeatable texture. Finally, after computing retargeted actions, we render the robot model executing the retargeted actions to obtain accurate link poses from the egocentric camera, and composite it with the synthesized background image. The same pipeline runs on real demonstrations, with the robot mask obtained from a rendered URDF depth buffer and intrinsics taken from the ZED2’s calibration (Appendix A).

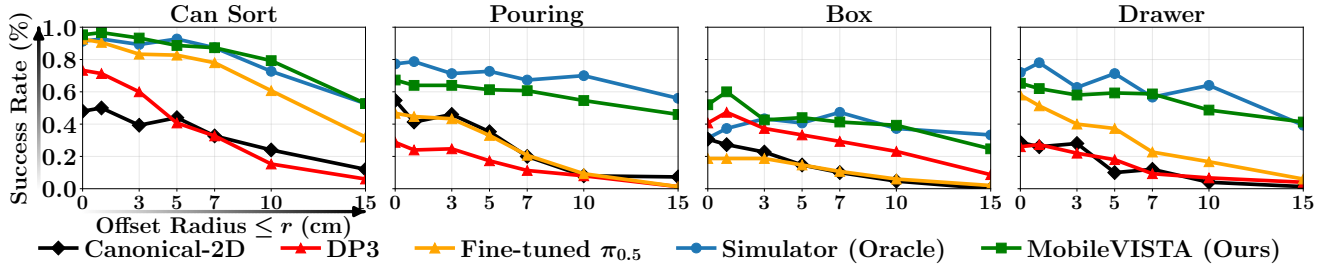


Fig. 3: **Randomized pose perturbation results.** Success rate as a function of evaluation perturbation radius r_{eval} (with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$). MobileVISTA maintains higher success rates than all baselines as perturbation magnitude grows.

It is fully automatic, with no manual filtering at any stage.

B. Retargeting Actions

We retarget actions to compensate for robot base pose perturbations in end-effector space. When source actions are joint-space, they are first converted into robot base frame end-effector goals via forward kinematics. We then apply the inverse of pose perturbation to each per-timestep end-effector goal, approximately preserving the *original* end-effector trajectory that is known to successfully complete the task. Where required, retargeted end-effector goals are converted back into joint-space actions via whole-body inverse kinematics. End-effector actuation (e.g., gripper or finger commands) is left unchanged.

Simulated demonstrations are recorded as end-effector poses, making retargeting closed-form, while JoyLo teleoperation records joint-space commands, which require an IK solve. The IK solve enforces smoothness, velocity bounds, joint limits, and self-collision avoidance (Appendix B). Past the initial transient, the retargeted trajectory tracks the original world-frame end-effector path, which is collision-free by construction. Within the sampled 15cm range all trajectories converged, so we apply no filtering. Under significant perturbations, IK failure would largely coincide with the task being unreachable from that base pose—a case for repositioning rather than augmentation. As in behavior cloning generally, we assume task-relevant contact and object dynamics are approximately unchanged under the sampled perturbations.

IV. EXPERIMENTS

In this section, we perform empirical analyses to answer the following questions:

Q1: Does MobileVISTA improve robustness to base poses outside the training distribution at test time, across humanoid and non-humanoid embodiments?

Q2: Are both coupled visual-action augmentation and explicit robot re-rendering at retargeted configurations necessary for pose generalization?

Q3: Does MobileVISTA outperform learned novel view synthesis for policy pose robustness?

Q4: Is MobileVISTA effective when deployed on real robotic hardware?

We compare MobileVISTA with the following baselines and prior methods:

- **Canonical-2D.** Trains diffusion policies [29] on 2D image observations from the original single canonical base pose only, without pose or viewpoint augmentation.

- **Simulator (Oracle).** Relocates the robot base to the sampled perturbed pose in simulation, retargets actions as in Section III-B, and records the egocentric observations from executing these actions. This yields ground-truth observations exactly consistent with the retargeted actions, upper-bounding MobileVISTA by testing perfect synthesis. This has no real-world counterpart, as re-recording demonstrations at every perturbed pose is not tractable at scale.
- **DP3 [30].** Trains policies on explicit 3D scene representations (e.g., point clouds) rather than RGB images, isolating the effect of geometry-aware inputs on pose robustness. Like Canonical-2D, DP3 trains only on canonical demonstration data, differing solely in input representation.
- **Fine-tuned $\pi_{0.5}$ [1].** Fine-tunes pretrained $\pi_{0.5}$ policy on each task’s canonical demonstration data, without pose augmentation or action retargeting, testing whether large-scale pretraining alone suffices for pose generalization.

Note that MobileVISTA is a data generation framework, not a policy architecture: MobileVISTA, Canonical-2D, and every ablation uses *identical* Diffusion Policy [29] architecture, hyperparameters (Appendix Table VII) and pipeline, differing only in training data, with no architectural change needed to consume augmented data. DP3 and fine-tuned $\pi_{0.5}$ instead test whether an alternative input representation alone confers pose robustness. Simulator (Oracle) matches MobileVISTA in architecture, data volume, sampled offsets, and training steps. Throughout, we use $r_{train} = r_{eval} = 15\text{cm}$ and $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$, chosen to span realistic post-navigation base error [4]. Source-to-augmented ratios are 1:1 in simulation and 1:10 in real, and each simulated result is a mean over three seeds at 50 rollouts.

A. Simulated Experimental Setup

We use the robomimic framework with the MuJoCo simulator [31] on tasks from DexMimicGen [32]. Can Sort trains on an even split of expert demonstrations across can color; the remaining tasks train on the first 200 demonstrations and evaluate on the DexMimicGen environment variants. Can Sort and Pouring tasks are performed on a Fourier GR-1 humanoid platform, while Box and Drawer tasks are evaluated on a bimanual Franka setup. Note that our simulated experiments serve to isolate synthesis quality against an oracle requiring simulator access, which has no real hardware

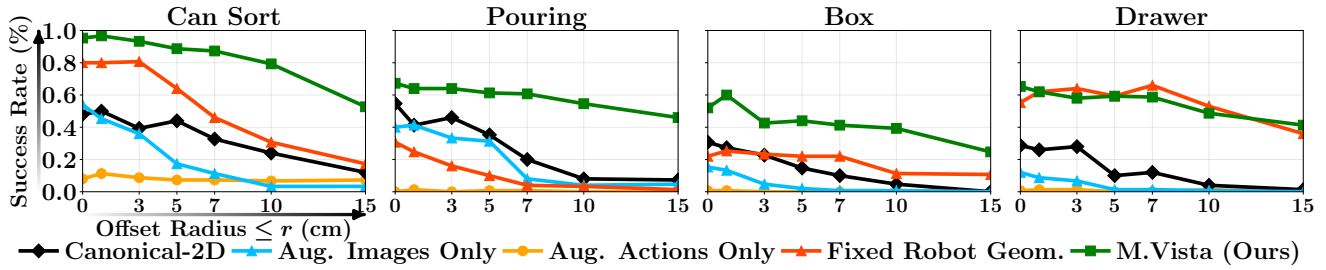


Fig. 4: **Ablation results across tasks.** Success rate vs. evaluation perturbation radius r_{eval} . We compare MobileVISTA against single-modality augmentation—*Aug. Images Only*, *Aug. Actions Only*— and an augmentation that omits robot re-rendering at retargeted configurations, *Fixed Robot Geom.* MobileVISTA outperforms nearly all ablated variants, indicating that both coupled visual–action augmentation and explicit robot re-rendering contribute to pose robustness.

equivalent. The benefit of augmenting over manual collection is tested on real hardware (Section IV-D).

Per trial, the robot base is displaced by an offset sampled uniformly within a disk of radius r_{eval} about the canonical pose, with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$ held fixed across all r_{eval} , including $r_{eval} = 0$. We sweep performance across radius sampling range $r_{eval} \in [0\text{cm}, 15\text{cm}]$, evaluating performance on progressively larger out-of-distribution base perturbations.

Q1: Robustness to Out-of-Distribution Base Poses. Results are shown in Figure 3. MobileVISTA consistently outperforms all baselines as the randomization range grows. In contrast, Canonical-2D training degrades rapidly as r increases, averaging under 45% success at $r_{eval} = 3\text{cm}$ across all tasks. DP3 and fine-tuned $\pi_{0.5}$ match or modestly improve on Canonical-2D in most cases, but demonstrate similarly limited generalization as perturbation magnitude increases. Notably, fine-tuned $\pi_{0.5}$ achieves robustness close to MobileVISTA and Simulator (Oracle) in the Can Sort task for offsets less than $r_{eval} = 5\text{cm}$. Overall, Canonical 2D training, explicit 3D representations, and large pretrained initialization are each insufficient on their own for robust pose generalization. These trends hold on both the humanoid GR-1 (Can Sort, Pouring) and the non-humanoid bimanual Franka (Box, Drawer). Full tabled results are in Appendix Section E.

B. Ablation Studies

Q2: Contribution of MobileVISTA’s Components. We ablate key components of MobileVISTA to understand their contributions. We evaluate whether: (1) *coupled* visual–action augmentation is necessary for pose generalization, and (2) explicitly re-rendering the robot at retargeted joint configurations is required to synthesize in-distribution egocentric observations.

1) *Coupled Visual–Action Augmentation:* Some augmentation frameworks [5]–[7] perturb visual observations in isolation, assuming the action modality can be treated as fixed. To test whether this suffices for mobile manipulation, we pair the MobileVISTA-augmented data from Section IV-A with its canonical source data, preserving demonstration order: *Aug. Actions Only* pairs canonical observations with retargeted actions, and *Aug. Images Only* pairs augmented observations with the original canonical actions, so exactly one modality is augmented relative to its source demon-

stration. Results are shown in Figure 4. We find that augmenting either modality in isolation consistently degrades performance across tasks, showing that jointly augmenting visual observations and actions is critical.

2) *Robot-Consistent Rendering After Retargeting:* MobileVISTA explicitly re-renders the robot at retargeted joint configurations, compositing this overlay onto the reprojected environment to ensure visual observations remain consistent with executed actions. While Section IV-B.1 examines whether both modalities must be augmented, here we examine whether the visual branch must account for the robot’s *own* motion under the base shift, rather than treating its geometry as static. We ablate our pipeline’s robot-consistent rendering by omitting the robot segmentation and rendering overlay of Section III-A: the *entire* scene is re-projected from depth to the target view without segmenting the robot out, treating it as static in the environment rather than coupled with the egocentric camera perturbation. Results are shown in Figure 4. Performance drops substantially under this setting in most tasks, suggesting that explicitly rendering accurate robot geometry after retargeting is critical for maintaining in-distribution egocentric observations.

When does robot-consistent rendering matter? Averaged over radii, MobileVISTA’s gap over *Fixed Robot Geom.* is largest on Pouring (+0.47), smaller on Can Sort and Box (+0.28 and +0.24), and absent on Drawer (≈ 0.00). We postulate robot-consistent rendering matters most under two task properties: how much of the egocentric frame the robot’s geometry occupies, and whether the camera rides the kinematic chain. Pouring, on the humanoid GR-1 with its actuated waist, has both and shows the largest gap, while the effect is weakest on the Franka tasks, where the rig-mounted camera sees less of the robot. This remains a hypothesis, as four tasks cannot separate the two properties and do not account for the Box–Drawer difference.

C. Quality Analysis of Synthesized Observations

Q3: Comparison of MobileVISTA with learned novel view synthesis. MobileVISTA generates pose-consistent observations from explicit geometry; learned novel view synthesis (NVS) is an alternative. We compare MobileVISTA-generated egocentric views against those produced by a representative NVS model via controlled PSNR evaluation. We render oracle views from the simulator at perturbed base

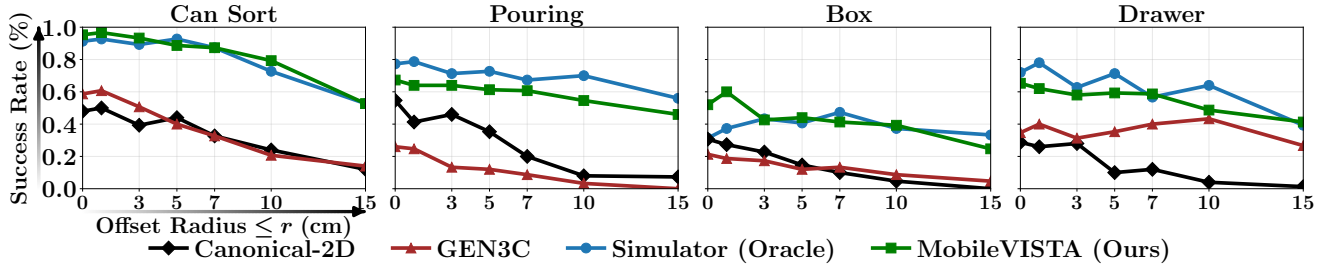


Fig. 5: **Policy performance under MobileVISTA vs. GEN3C-synthesized training data.** Success rate vs. evaluation perturbation radius r_{eval} , alongside the Simulator (Oracle) upper bound. Policies trained on MobileVISTA-synthesized data outperform those trained on GEN3C-synthesized data across tasks and perturbation magnitudes.

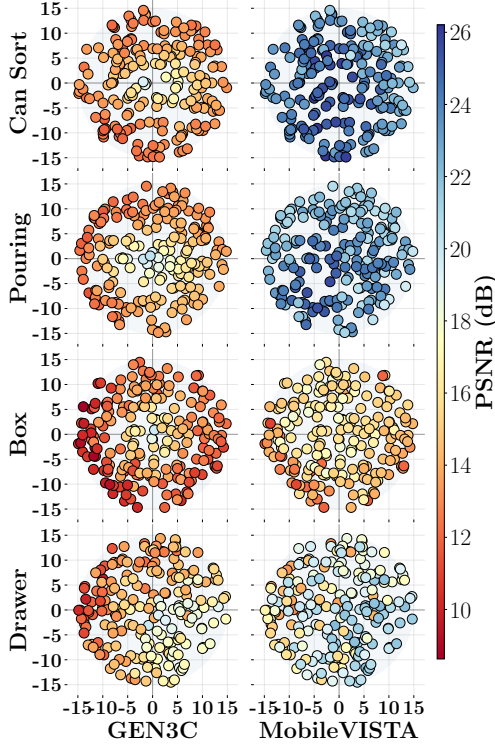


Fig. 6: **Per-trajectory PSNR for synthesized images.** Each point shows the PSNR ($\uparrow$, dB) of a synthesized trajectory at its sampled base offset from canonical (cm, overhead view). MobileVISTA yields higher PSNR overall, with the largest separation on humanoid tasks. Tasks where the methods are closer in PSNR also show smaller gaps in downstream policy performance.

poses and treat these as ground truth, then measure PSNR between these oracle images and views synthesized by (i) MobileVISTA, and (ii) GEN3C [11], a leading learned NVS model for 3D-consistent camera-controlled video generation, under identical camera perturbations.

We train policies on the GEN3C views with the same action retargeting (Section III-B), and compare performance against MobileVISTA and oracle policies of Section IV-A.

As shown in Figure 6, MobileVISTA consistently achieves higher PSNR across perturbation magnitudes, indicating more consistent preservation of scene geometry and object structure. In contrast, GEN3C synthesis degrades noticeably under larger viewpoint shifts, often producing blurred object boundaries or distorted geometry. The gap is not uniform across embodiments: at offsets of 10–15cm, MobileVISTA

exceeds GEN3C by 10.2dB and 8.9dB on the two humanoid GR-1 tasks, against 2.9dB and 3.9dB on the two Franka tasks (Appendix Table V). Results from policy learning are presented in Figure 5. We find that policies trained on MobileVISTA-synthesized data outperform those trained on GEN3C across tasks and perturbation magnitudes, consistent with MobileVISTA’s higher reconstruction fidelity.

Additionally, MobileVISTA generates augmented data $4\text{--}7\times$ faster than GEN3C across tasks, with full per-task averages tabled in Appendix Table VI. Full tabled PSNR values, as well as qualitative comparisons to further highlight these effects, can be found in Appendix Section F.

Beyond reconstruction quality, NVS-based approaches treat the robot as part of the static scene, limiting how well it reflects base-conditioned changes in robot configuration. Prior work [5] showed promising NVS results for externally mounted cameras observing static robots; we postulate the static-robot assumption is especially problematic in egocentric settings, where camera motion and robot articulation are tightly coupled. MobileVISTA instead re-renders the robot under retargeted actions and composites it with a geometry-consistent background.

D. Real-World Evaluation

Q4: Deployment on real hardware. We investigate the performance of MobileVISTA on real-robot data using a Galaxea R1 Pro humanoid. We evaluate on two representative real-world manipulation tasks:

- **Laundry Sort:** the robot must sort four laundry items into two bins based on category. This task is challenging due to its long horizon, the deformability of the clothing items, and the bimanual coordination required to sort multiple objects in sequence.
- **Book Stacking:** the robot must grasp a vertically placed book and reorient it horizontally onto an existing stack inside a bookshelf. This is challenging due to the precise lateral alignment required for a successful grasp.

We collect 50 expert demonstrations per task from a non-perturbed robot pose using the JoyLo framework introduced in BEHAVIOR Robot Suite [33], and train a MobileVISTA policy and baselines using the same framework as described in Section III.

Evaluation procedure. For each trial, we sample a base offset uniformly within $r_{eval} = 15\text{cm}$ of the canonical pose and manually drive the robot to that offset. Yaw is

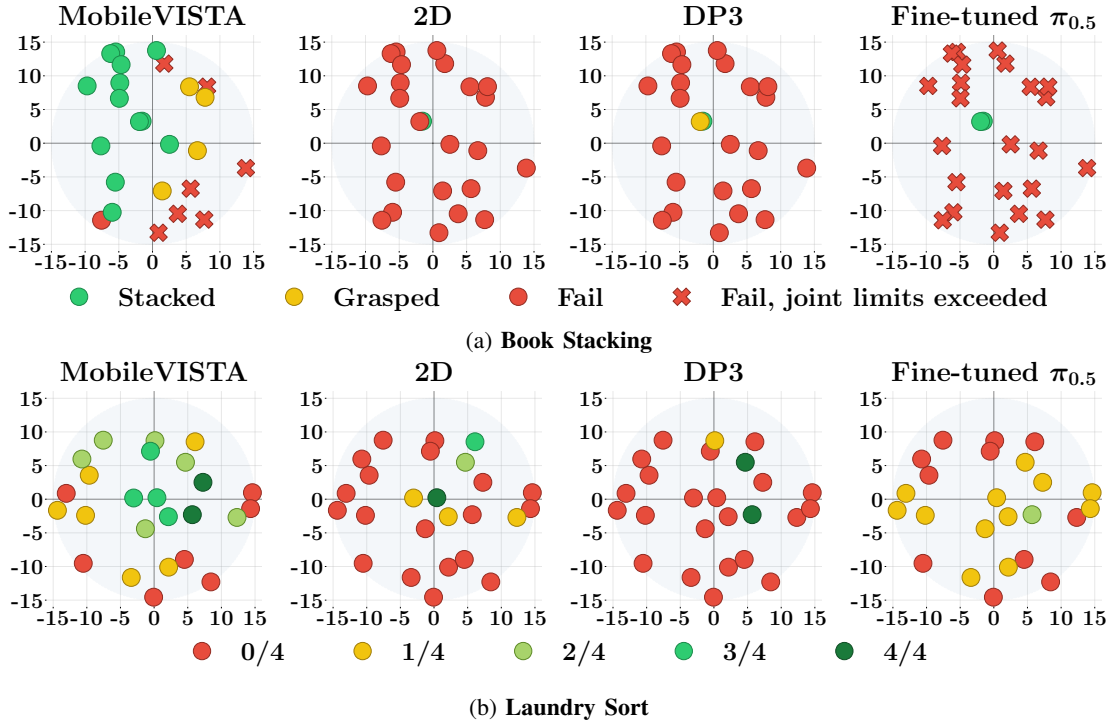


Fig. 7: **Sampled offsets at $r_{eval} = 15\text{cm}$ in real-world evaluation.** Outcomes for each policy across sampled base offsets (cm, overhead view), for book stacking (a) and laundry sort (b). MobileVISTA succeeds across a wide range of offsets, while baselines fail to generalize beyond the canonical pose.

not explicitly controlled, as policies are trained over the full $\pm 5^\circ$ yaw range used during augmentation. From each sampled offset, we roll out all four policies (MobileVISTA, Canonical-2D, DP3, Fine-Tuned $\pi_{0.5}$) without repositioning the robot. The sampled offset is held fixed across all four policies with no repositioning between them, so docking stochasticity cannot advantage any method, and policy evaluation order at each offset is randomized and unknown to the experimenter scoring the rollout. We score each Book Stacking rollout in two stages: *Grasped*, where the robot successfully grasps the book, and *Stacked*, where the book is successfully placed onto the existing stack.

Results. Full success rates are reported in Table II and the spatial distribution of outcomes at $r_{eval} = 15\text{cm}$ in Figure 7. On Book Stacking, the MobileVISTA-trained policy grasps in 17/25 randomized trials and stacks in 13/25, while no baseline exceeds 2/25 at either stage.

MobileVISTA generalizes particularly well to leftward and forward offsets; its remaining failures are predominantly joint-limit violations when reaching for the book, which we postulate arise when the retargeted trajectory requires the arm to cross the torso. In contrast, Canonical-2D, DP3, and Fine-Tuned $\pi_{0.5}$ execute the original demonstration trajectory without compensating for the new base pose: forward offsets cause collisions with the shelf, backward offsets leave the gripper short of the book. Fine-Tuned $\pi_{0.5}$ also consistently outputs actions outside the robot’s joint limits, though its qualitative failure modes resemble Canonical-2D and DP3.

On Laundry Sort, MobileVISTA sorts at least one item in 18/25 trials and at least two in 12/25; the strongest baselines reach 14/25 (fine-tuned $\pi_{0.5}$) and 3/25 (Canonical-

Laundry Sort				
Method	≥ 1	≥ 2	≥ 3	≥ 4
<i>Randomized Pose</i>				
Canonical-2D	6/25	3/25	2/25	1/25
DP3	3/25	2/25	2/25	2/25
Fine-tuned $\pi_{0.5}$	14/25	2/25	1/25	1/25
MobileVISTA (Ours)	18/25	12/25	6/25	2/25
<i>Canonical Pose</i>				
Canonical-2D	21/25	15/25	5/25	3/25
DP3	19/25	11/25	8/25	3/25
Fine-tuned $\pi_{0.5}$	21/25	8/25	7/25	5/25
MobileVISTA (Ours)	24/25	15/25	9/25	4/25

Book Stacking		
Method	Grasped	Stacked
<i>Randomized Pose</i>		
Canonical-2D	1/25	1/25
DP3	2/25	1/25
Fine-tuned $\pi_{0.5}$	2/25	2/25
MobileVISTA (Ours)	17/25	13/25
<i>Canonical Pose</i>		
Canonical-2D	25/25	24/25
DP3	20/25	8/25
Fine-tuned $\pi_{0.5}$	25/25	25/25
MobileVISTA (Ours)	19/25	18/25

TABLE II: **Real-world success rates ($\uparrow$) for book stacking and laundry sort under canonical and randomized base poses.** For laundry sort, each cell reports the number of trials (of 25) in which at least N items were correctly sorted. For book stacking, we report grasp and stack success in 25 trials. $r_{eval} = 15\text{cm}$ for randomized. MobileVISTA maintains strong performance under base randomization, while baselines collapse despite strong canonical-pose performance.

2D). Failure modes separate by method and direction. Qualitatively, Canonical-2D and DP3 policies do not compensate

for backward shifts, stopping short when attempting to grasp items. Canonical-2D additionally executes the bimanual handoff motion with nothing in hand after a missed grasp; neither mode appears in any MobileVISTA rollout. Fine-tuned $\pi_{0.5}$ fails predominantly by contact with the table and bins, and rarely progresses past a first item. MobileVISTA also contacts the table and bins, but at backward offsets, over-shooting the items rather than stopping short of them. At the canonical pose on Book Stacking, MobileVISTA grasps in 19/25 and stacks in 18/25 against 25/25 and 24–25/25 for Canonical-2D and fine-tuned $\pi_{0.5}$, a precision cost absent on Laundry Sort, where MobileVISTA is best or tied-best at three of the four thresholds. Overall, we find that MobileVISTA generalizes to perturbed base poses on real hardware more robustly than canonical training, explicit 3D representations, and large pretrained policies, with successful rollouts distributed across the sampled offset range rather than concentrated near the demonstration pose.

V. CONCLUSION & LIMITATIONS

Limitations. While MobileVISTA demonstrates strong viewpoint-robust policy learning, several limitations remain. First, synthesis quality depends on camera field-of-view: narrower FOVs reduce visible context, potentially degrading the consistency of generated views and policy performance. Second, we augment only egocentric cameras and not wrist-mounted cameras, leaving potential robustness gains from multi-view augmentation unexplored. Third, all results are bounded by the sampled perturbation range (15cm translation, $\pm 5^\circ$ yaw), and behavior beyond it is untested.

Conclusion. We presented MobileVISTA, a data generation framework that improves the robustness of mobile manipulation policies to base pose perturbations by jointly synthesizing pose-perturbed egocentric observations and re-targeted actions from single-pose demonstrations. Unlike prior augmentation methods, which assume either a camera mounted off the actuated chain or non-trivial articulated robot geometry out of frame, our evaluation includes settings where neither assumption holds: the camera rides the actuated chain, and that chain occupies much of the observation. Experiments in both simulation and on a real Galaxea R1 Pro humanoid demonstrated MobileVISTA’s improved robustness over strong baselines. Promising future directions include integrating MobileVISTA with large pretrained policies and extending augmentation to wrist-mounted cameras.

REFERENCES

- [1] P. Intelligence *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” 2025.
- [2] NVIDIA *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” in *ArXiv Preprint*, March 2025.
- [3] G. A. Team, “Gen-0: Embodied foundation models that scale with physical interaction,” *Generalist AI Blog*, 2025.
- [4] J. Yang *et al.*, “Mobi- π : Mobilizing your robot learning policy,” in *Conference on Robot Learning (CoRL)*, 2025.
- [5] S. Tian *et al.*, “View-invariant policy learning via zero-shot novel view synthesis,” *Conference on Robotic Learning (CoRL)*, 2024.
- [6] J. Chen *et al.*, “Ropa: Synthetic robot pose generation for rgb-d bimanual data augmentation,” *arXiv preprint arXiv:2509.19454*, 2025.
- [7] L. Y. Chen *et al.*, “Rovi-aug: Robot and viewpoint augmentation for cross-embodiment robot learning,” in *Conference on Robot Learning (CoRL)*, Munich, Germany, 2024.
- [8] Y. Xu *et al.*, “EgoDemoGen: Egocentric demonstration generation for viewpoint generalization in robotic manipulation,” *arXiv preprint arXiv:2509.22578*, 2025.
- [9] X. Xu *et al.*, “R2RGen: Real-to-real 3d data generation for spatially generalized manipulation,” *arXiv preprint arXiv:2510.08547*, 2025.
- [10] C. Pan *et al.*, “One demo is worth a thousand trajectories: Action-view augmentation for visuomotor policies,” in *Conference on Robot Learning (CoRL)*, 2025.
- [11] X. Ren *et al.*, “Gen3c: 3d-informed world-consistent video generation with precise camera control,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [12] H. Ha *et al.*, “UMI on legs: Making manipulation policies mobile with manipulation-centric whole-body controllers,” in *Proceedings of the 2024 Conference on Robot Learning*, 2024.
- [13] Z. Fu *et al.*, “Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation,” in *Conference on Robot Learning (CoRL)*, 2024.
- [14] R. Mendonca *et al.*, “Continuously improving mobile manipulation with autonomous real-world rl,” 2024.
- [15] J. Gu *et al.*, “Multi-skill mobile manipulation for object rearrangement,” 2022.
- [16] J. Wu *et al.*, “Tidybot: Personalized robot assistance with large language models,” *Autonomous Robots*, 2023.
- [17] N. Yokoyama *et al.*, “Asc: Adaptive skill coordination for robotic mobile manipulation,” *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 779–786, 2023.
- [18] Z. Wu *et al.*, “Moto: A zero-shot plug-in interaction-aware navigation for general mobile manipulation,” 2025.
- [19] K. Chai *et al.*, “N2m: Bridging navigation and manipulation by learning pose preference from rollout,” 2025.
- [20] Z. Mandi *et al.*, “CACTI: A framework for scalable multi-task multi-scene visual imitation learning,” *arXiv:2212.05711*, 2022.
- [21] T. Yu *et al.*, “Scaling robot learning with semantically imagined experience,” in *Robotics: Science and Systems (RSS)*, 2023.
- [22] Z. Chen *et al.*, “Genaug: Retargeting behaviors to unseen situations via generative augmentation,” *arXiv preprint arXiv:2302.06671*, 2023.
- [23] E. Ameperosa *et al.*, “Rocoda: Counterfactual data augmentation for data-efficient robot learning from demonstrations,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 13 250–13 256.
- [24] Z. Xue *et al.*, “Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning,” *arXiv preprint arXiv:2502.16932*, 2025.
- [25] M. Lepert *et al.*, “Phantom: Training robots without robots using only human videos,” 2025.
- [26] I.-C. A. Liu *et al.*, “D-coda: Diffusion for coordinated dual-arm data augmentation,” in *Conference on Robot Learning (CoRL)*, 2025.
- [27] S. Yang *et al.*, “Novel demonstration generation with gaussian splatting enables robust one-shot manipulation,” *arXiv preprint arXiv:2504.13175*, 2025.
- [28] Z. Li *et al.*, “Towards an end-to-end framework for flow-guided video inpainting,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [29] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [30] Y. Ze *et al.*, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [31] E. Todorov *et al.*, “Mujoco: A physics engine for model-based control,” in *Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [32] Z. Jiang *et al.*, “Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [33] Y. Jiang *et al.*, “BEHAVIOR robot suite: Streamlining real-world whole-body manipulation for everyday household activities,” in *Conference on Robot Learning (CoRL)*, 2025.
- [34] K. Zakka, “mink: Python inverse kinematics based on MuJoCo,” 2024.
- [35] A. Mandlekar *et al.*, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.

In addition to the following information, please see the supplementary video and project website (<https://sdwistreich.github.io/mobilevista/>) for a visual walk-through of the framework and example rollout videos from each policy on each task.

A. Visual Observation Synthesis

1) *Robot Model and Segmentation*: We implement visual observation synthesis using MuJoCo via the robosuite framework. For each task, we render the robot using the same MuJoCo model used to instantiate the source demonstration, with non-robot geoms in the scene rendered invisible to produce a clean robot silhouette.

To extract a 2D mask of the robot, we use a different procedure across the simulated and real-world settings. In simulated experiments, we modify the scene’s skybox color to a distinctive background color (red for the GR-1 humanoid setting, yellow for the bimanual Franka setting, the latter chosen to avoid conflicts with red environment objects), render the robot with all other geoms hidden, and threshold pixels by background color to produce a binary mask. In the real-world setting, we instead render the R1 Pro robot model in MuJoCo from a URDF specification and extract the robot silhouette directly from the resulting depth buffer (thresholded at 0.995 to exclude background), then dilate the resulting mask by 5 pixels to capture edge artifacts.

We identify robot geoms in MuJoCo by body name prefixes: `robot0/gripper0` (and `robot1/gripper1` in the bimanual setting) for simulation, and the robot’s named body geoms (e.g., `left_arm_link*`, `torso_link*`) for the real-world R1 Pro setup. Rendering is performed at 512×512 resolution for simulated experiments and at the native dataset resolution for real-world experiments, upsampled by a factor of $6 \times$ during point cloud reprojection.

2) *Depth Reprojection and Camera Extrinsic*s: We lift the masked RGB-D observation into a colored point cloud using a pinhole camera model with intrinsics derived from the camera’s vertical field of view (`cam_fovy` in MuJoCo). For real-world data, we use the ZED2 camera’s calibrated intrinsics ($f_x = 58.4$, $f_y = 58.5$, $c_x = 74.7$, $c_y = 42.9$ at the dataset resolution, scaled with image dimensions during upsampling). Point clouds are constructed using Open3D’s `RGBDImage.create_from_color_and_depth` with depth scale 1000mm and a depth truncation of 5m (simulation) or 10m (real).

To lift the point cloud into the world frame, we use the camera-to-world extrinsics obtained from MuJoCo at the canonical configuration (`cam_xpos`, `cam_xmat`). After applying the base perturbation Δp , we re-render the robot model at its retargeted joint configuration and obtain new camera extrinsics from the resulting kinematic chain. We reproject the world-frame point cloud onto the new camera using an Open3D `OffscreenRenderer`, producing a pose-shifted RGB image and a corresponding hole mask from pixels not covered by reprojected points.

3) *Inpainting*: To fill missing regions in the reprojected RGB image, including both regions originally occupied by the segmented-out robot and newly disoccluded areas introduced by the base shift, we apply video inpainting with E2FGVI [28]. We process each trajectory’s frames and per-frame hole masks as a temporal batch via E2FGVI’s official pipeline. Prior to inpainting, we apply a 3×3 morphological closing (3 iterations) to the hole mask to fill small gaps introduced by point cloud sparsity. We use the model checkpoint `E2FGVI-HQ-CVPR22.pth`.

4) *Robot Re-Compositing*: After inpainting, we re-render the robot model at its retargeted joint configuration using MuJoCo, with non-robot geoms hidden as before. We composite the re-rendered robot onto the inpainted background by masking robot pixels from the rendered image. In the simulation pipeline, we use a color-based mask (excluding pixels matching the masking skybox color) to handle background bleed-through. In the real-world pipeline, we use the depth buffer from the re-rendered robot to identify robot pixels, again dilated by 5 pixels to capture edges. After compositing, real-world outputs are downsampled back to the original dataset resolution.

B. Action Retargeting

1) *Forward and Inverse Kinematics*: In the simulated humanoid (GR-1) and bimanual Franka settings, actions are originally represented as per-arm end-effector poses (3D position + 3D rotation vector per arm) in the robot base frame. Retargeting is performed in closed form by applying the inverse base perturbation transform to each arm’s end-effector goal at each timestep (Section B.2). In the bimanual setting, the rig caches both robot base poses and the shared camera pose; the yaw rotation is applied about the midpoint of the two base positions to preserve their relative configuration.

In the real-world (R1 Pro) setting, we instead solve for new joint configurations using whole-body inverse kinematics. We use the `mink` [34] IK solver with the following setup:

- **FrameTasks** for the left and right end-effector sites (`left_ee_center`, `right_ee_center`), each with position cost and orientation cost set to 1.0.
- **PostureTask** regularizing toward the previous configuration, with cost 10^{-5} .
- **ConfigurationLimit** on joint positions.
- **VelocityLimit** with a per-step bound of 1.0 on arm joints. The torso joint limit is set to 0.02 for Book Stacking and 0.04 for Laundry Sort, preferring arm motion over torso motion to maintain a stable view-point while allowing slightly more torso motion for the longer-horizon Laundry Sort task.
- **CollisionAvoidanceLimit** between end-effector and torso geom groups with a minimum collision distance of 0.04m and a detection distance of 0.18m.

We iterate the IK solver for up to 20 steps per frame with $\Delta t = 0.01$ using `quadprog` as the underlying solver, with damping 10^{-3} and a position tolerance of 0.01m for convergence.

2) *End-Effector Goal Perturbation*: Given a base perturbation $\Delta p = (\Delta b, \Delta \theta)$ with translation $\Delta b \in \mathbb{R}^2$ and yaw $\Delta \theta$, we construct the inverse transform $T_{\Delta p}^{-1}$ as a 4×4 matrix combining the inverse yaw rotation and a translation. For each arm’s per-timestep end-effector position $\mathbf{p}_t$ (in the robot’s base frame), the compensated position is $T_{\Delta p}^{-1} \cdot [\mathbf{p}_t; 1]$. Orientations, represented as rotation vectors, are updated by composing with the inverse yaw rotation: $R'_t = R_{\text{inv yaw}} \cdot R(\mathbf{r}_t)$, then converted back to a rotation vector. Gripper and finger commands are passed through unchanged.

For the bimanual Franka setting, the retargeting accounts for each arm’s new base position after yaw is applied about the rig midpoint, so the per-arm compensation includes both a rotation and a translation derived from the change in that arm’s base pose.

For the real-world R1 Pro setting, the end-effector position and orientation targets passed to IK at $t = 0$ are obtained by transforming the original end-effector poses around the rig pivot under Δp . For subsequent timesteps, IK is solved to track the original end-effector trajectory directly, allowing the arm to bend differently to reach the same world-frame target from the new base pose.

C. Augmentation Sampling

For each augmented trajectory, we sample a base translation offset Δb uniformly within a disk of radius $r_{\text{train}} = 15\text{cm}$ centered at the canonical pose using the standard formulation $(r, \theta) \sim (\sqrt{u} \cdot r_{\text{train}}, 2\pi v)$ for $u, v \sim \mathcal{U}(0, 1)$, with $\Delta b = (r \cos \theta, r \sin \theta, 0)$. No vertical offset is applied. When yaw jitter is enabled, we additionally sample $\Delta \theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. We use the same training and evaluation radius $r_{\text{train}} = r_{\text{eval}} = 15\text{cm}$ across all simulated and real-world tasks.

For simulated tasks, we generate one augmented trajectory per source demonstration, yielding 200 augmented trajectories from 200 source demonstrations. For real-world tasks, we generate ten augmented trajectories per source demonstration, yielding 500 augmented trajectories from 50 source demonstrations. We do not filter augmented trajectories based on IK convergence; in practice, the IK solver converges within tolerance for the vast majority of frames.

D. Task Definitions

We evaluate on four tasks from DexMimicGen [32]. For the Can Sort task, we use an even split of expert demonstrations across can color (blue, red) for training. For the remainder of the tasks, we train using the first 200 demonstrations of the datasets and evaluate using the environment definitions provided by DexMimicGen, detailed below. Can Sort and Pouring are performed on a Fourier GR-1 humanoid, and Box and Drawer are performed on a bimanual Franka setup.

- **Can Sort**: The robot must pick up a can with one arm, perform a handover to the other arm, and deposit it into the correct bin based on color. The handover phase is a coordination subtask requiring both arms to cooperate with synchronized trajectories. Can color and placement

are randomized according to DexMimicGen defaults; the target bins are fixed.

- **Pouring**: A sequential task in which the robot must first pour a ball from a cup into a bowl using one arm, then pick up the bowl with the other arm and place it onto a green pad. The cup and bowl positions are randomized; the pad location is fixed.
- **Box**: The robot must pick up a box lid and place it onto the box using both arms simultaneously, requiring precise relative alignment between the two end-effectors. The positions of the box and lid are randomized.
- **Drawer**: A sequential task in which the robot must open a drawer with one arm before placing a mug inside with the other. The mug location is randomized according to DexMimicGen defaults.

Camera setup. For the bimanual Franka tasks (Box, Drawer), we use the `agentview` camera, a fixed rigid camera positioned between the two arms to approximate an egocentric viewpoint. For the GR-1 humanoid tasks (Can Sort, Pouring), we use the `robot0_robotview` head-mounted camera. We widen its field of view from the DexMimicGen default of 50° to 90° to ensure the full manipulation workspace is visible.

E. Tabled Per-Task Results

Randomized Pose Perturbations. Table III reports full per-task success rates for all methods under the randomized base pose perturbation regime from Section IV-A (Figure 3) of the main paper. Each cell reports mean success rate and standard deviation over three random seeds.

Ablations. Table IV reports full per-task ablation results corresponding to Figure 4 of the main paper. Each cell reports mean $\pm$ std over three random seeds. MobileVISTA wins across nearly all tasks and perturbation magnitudes. The exception is the Drawer task, where Fixed Robot Geom. is competitive or superior at intermediate perturbation radii, suggesting that this task may be less sensitive to inaccurate robot re-rendering.

F. Comparison of MobileVISTA with Learned Novel View Synthesis

We provide additional details supporting the comparison between MobileVISTA and the learned novel view synthesis baseline GEN3C [11] introduced in Section IV-C. We report quantitative reconstruction quality, qualitative side-by-side comparisons, generation throughput, and downstream policy learning performance when training on views synthesized by each method.

Data Generation. We integrate GEN3C [11] into our pipeline as a drop-in replacement for the geometry-based reprojection step in MobileVISTA, while keeping the action retargeting and robot re-compositing components unchanged. To ensure a fair comparison, we make several modifications to the default GEN3C setup.

First, rather than estimating depth and camera parameters from RGB using VIPE as in the official GEN3C implementation, we seed GEN3C with ground-truth depth and ground-

Task	Method	$r=0$	$r=1$	$r=3$	$r=5$	$r=7$	$r=10$	$r=15$
Can Sort	Canonical-2D	0.480 $\pm$ 0.02	0.500 $\pm$ 0.09	0.393 $\pm$ 0.04	0.440 $\pm$ 0.05	0.327 $\pm$ 0.03	0.240 $\pm$ 0.00	0.120 $\pm$ 0.06
	DP3	0.733 $\pm$ 0.08	0.713 $\pm$ 0.06	0.600 $\pm$ 0.13	0.407 $\pm$ 0.05	0.327 $\pm$ 0.06	0.153 $\pm$ 0.04	0.060 $\pm$ 0.02
	Fine-tuned $\pi_{0.5}$	0.926 $\pm$ 0.05	0.906 $\pm$ 0.01	0.833 $\pm$ 0.06	0.827 $\pm$ 0.10	0.780 $\pm$ 0.09	0.607 $\pm$ 0.09	0.320 $\pm$ 0.03
	MobileVISTA (Ours)	0.953$\pm$0.03	0.967$\pm$0.02	0.933$\pm$0.02	0.887$\pm$0.05	0.873$\pm$0.05	0.793$\pm$0.06	0.527$\pm$0.04
	<i>Simulator (Oracle)</i>	0.913 $\pm$ 0.02	0.927 $\pm$ 0.05	0.893 $\pm$ 0.01	0.927 $\pm$ 0.03	0.873 $\pm$ 0.05	0.727 $\pm$ 0.01	0.527 $\pm$ 0.08
Pouring	Canonical-2D	0.547 $\pm$ 0.03	0.413 $\pm$ 0.01	0.460 $\pm$ 0.08	0.353 $\pm$ 0.03	0.200 $\pm$ 0.09	0.080 $\pm$ 0.02	0.073 $\pm$ 0.06
	DP3	0.287 $\pm$ 0.08	0.240 $\pm$ 0.09	0.247 $\pm$ 0.06	0.173 $\pm$ 0.06	0.113 $\pm$ 0.03	0.080 $\pm$ 0.06	0.013 $\pm$ 0.01
	Fine-tuned $\pi_{0.5}$	0.467 $\pm$ 0.03	0.447 $\pm$ 0.02	0.433 $\pm$ 0.08	0.333 $\pm$ 0.05	0.207 $\pm$ 0.06	0.093 $\pm$ 0.03	0.013 $\pm$ 0.01
	MobileVISTA (Ours)	0.673$\pm$0.02	0.640$\pm$0.03	0.640$\pm$0.06	0.613$\pm$0.04	0.607$\pm$0.10	0.547$\pm$0.12	0.460$\pm$0.02
	<i>Simulator (Oracle)</i>	0.773 $\pm$ 0.03	0.787 $\pm$ 0.06	0.713 $\pm$ 0.13	0.727 $\pm$ 0.03	0.673 $\pm$ 0.03	0.700 $\pm$ 0.07	0.560 $\pm$ 0.11
Box	Canonical-2D	0.307 $\pm$ 0.11	0.273 $\pm$ 0.02	0.227 $\pm$ 0.09	0.147 $\pm$ 0.10	0.100 $\pm$ 0.03	0.047 $\pm$ 0.02	0.000 $\pm$ 0.00
	DP3	0.407 $\pm$ 0.01	0.473 $\pm$ 0.02	0.373 $\pm$ 0.05	0.333 $\pm$ 0.02	0.293 $\pm$ 0.05	0.213 $\pm$ 0.03	0.087 $\pm$ 0.05
	Fine-tuned $\pi_{0.5}$	0.187 $\pm$ 0.10	0.187 $\pm$ 0.06	0.187 $\pm$ 0.08	0.147 $\pm$ 0.01	0.107 $\pm$ 0.01	0.060 $\pm$ 0.03	0.020 $\pm$ 0.00
	MobileVISTA (Ours)	0.520$\pm$0.14	0.600$\pm$0.07	0.427$\pm$0.08	0.440$\pm$0.07	0.413$\pm$0.03	0.393$\pm$0.07	0.247$\pm$0.05
	<i>Simulator (Oracle)</i>	0.313 $\pm$ 0.09	0.373 $\pm$ 0.02	0.433 $\pm$ 0.10	0.407 $\pm$ 0.06	0.473 $\pm$ 0.05	0.373 $\pm$ 0.06	0.333 $\pm$ 0.01
Drawer	Canonical-2D	0.287 $\pm$ 0.10	0.260 $\pm$ 0.05	0.280 $\pm$ 0.00	0.100 $\pm$ 0.04	0.120 $\pm$ 0.03	0.040 $\pm$ 0.02	0.013 $\pm$ 0.01
	DP3	0.260 $\pm$ 0.00	0.273 $\pm$ 0.02	0.220 $\pm$ 0.05	0.180 $\pm$ 0.03	0.093 $\pm$ 0.05	0.067 $\pm$ 0.06	0.040 $\pm$ 0.02
	Fine-tuned $\pi_{0.5}$	0.580 $\pm$ 0.09	0.513 $\pm$ 0.05	0.400 $\pm$ 0.02	0.373 $\pm$ 0.11	0.227 $\pm$ 0.06	0.167 $\pm$ 0.08	0.060 $\pm$ 0.02
	MobileVISTA (Ours)	0.653$\pm$0.04	0.620$\pm$0.05	0.580$\pm$0.07	0.593$\pm$0.06	0.587$\pm$0.02	0.487$\pm$0.09	0.413$\pm$0.01
	<i>Simulator (Oracle)</i>	0.720 $\pm$ 0.03	0.780 $\pm$ 0.07	0.627 $\pm$ 0.03	0.713 $\pm$ 0.07	0.567 $\pm$ 0.05	0.640 $\pm$ 0.09	0.393 $\pm$ 0.03

TABLE III: Detailed per-task success rates under randomized pose perturbation across evaluation radii $r_{eval} \in \{0, 1, 3, 5, 7, 10, 15\}$ cm with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. Each cell reports the mean $\pm$ std over three random seeds (50 evaluation rollouts per seed). **Bold** indicates the best non-oracle method per row. Simulator (Oracle) is reported as a reference upper bound.

Task	Method	$r=0$	$r=1$	$r=3$	$r=5$	$r=7$	$r=10$	$r=15$
Can Sort	Canonical-2D	0.48 $\pm$ 0.02	0.50 $\pm$ 0.09	0.39 $\pm$ 0.04	0.44 $\pm$ 0.05	0.33 $\pm$ 0.03	0.24 $\pm$ 0.00	0.12 $\pm$ 0.06
	Aug. Images Only	0.54 $\pm$ 0.05	0.45 $\pm$ 0.09	0.36 $\pm$ 0.00	0.17 $\pm$ 0.10	0.11 $\pm$ 0.04	0.03 $\pm$ 0.02	0.03 $\pm$ 0.02
	Aug. Actions Only	0.08 $\pm$ 0.07	0.11 $\pm$ 0.08	0.09 $\pm$ 0.01	0.07 $\pm$ 0.06	0.07 $\pm$ 0.05	0.07 $\pm$ 0.04	0.07 $\pm$ 0.05
	Fixed Robot Geom.	0.80 $\pm$ 0.05	0.80 $\pm$ 0.07	0.81 $\pm$ 0.05	0.64 $\pm$ 0.05	0.46 $\pm$ 0.09	0.31 $\pm$ 0.10	0.17 $\pm$ 0.06
	MobileVISTA (Ours)	0.95 $\pm$ 0.03	0.97 $\pm$ 0.02	0.93 $\pm$ 0.02	0.89 $\pm$ 0.05	0.87 $\pm$ 0.05	0.79 $\pm$ 0.06	0.53 $\pm$ 0.04
Pouring	Canonical-2D	0.55 $\pm$ 0.03	0.41 $\pm$ 0.01	0.46 $\pm$ 0.08	0.35 $\pm$ 0.03	0.20 $\pm$ 0.09	0.08 $\pm$ 0.02	0.07 $\pm$ 0.06
	Aug. Images Only	0.40 $\pm$ 0.05	0.41 $\pm$ 0.04	0.33 $\pm$ 0.12	0.31 $\pm$ 0.08	0.08 $\pm$ 0.02	0.04 $\pm$ 0.04	0.05 $\pm$ 0.02
	Aug. Actions Only	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.31 $\pm$ 0.08	0.25 $\pm$ 0.01	0.16 $\pm$ 0.04	0.10 $\pm$ 0.07	0.04 $\pm$ 0.02	0.03 $\pm$ 0.01	0.01 $\pm$ 0.02
	MobileVISTA (Ours)	0.67 $\pm$ 0.02	0.64 $\pm$ 0.03	0.64 $\pm$ 0.06	0.61 $\pm$ 0.04	0.61 $\pm$ 0.10	0.55 $\pm$ 0.12	0.46 $\pm$ 0.02
Box	Canonical-2D	0.31 $\pm$ 0.11	0.27 $\pm$ 0.02	0.23 $\pm$ 0.09	0.15 $\pm$ 0.10	0.10 $\pm$ 0.03	0.05 $\pm$ 0.02	0.00 $\pm$ 0.00
	Aug. Images Only	0.15 $\pm$ 0.07	0.13 $\pm$ 0.08	0.05 $\pm$ 0.01	0.02 $\pm$ 0.00	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Aug. Actions Only	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.22 $\pm$ 0.07	0.25 $\pm$ 0.09	0.23 $\pm$ 0.02	0.22 $\pm$ 0.11	0.22 $\pm$ 0.05	0.11 $\pm$ 0.03	0.11 $\pm$ 0.03
	MobileVISTA (Ours)	0.52 $\pm$ 0.14	0.60 $\pm$ 0.07	0.43 $\pm$ 0.08	0.44 $\pm$ 0.07	0.41 $\pm$ 0.03	0.39 $\pm$ 0.07	0.25 $\pm$ 0.05
Drawer	Canonical-2D	0.29 $\pm$ 0.10	0.26 $\pm$ 0.05	0.28 $\pm$ 0.00	0.10 $\pm$ 0.04	0.12 $\pm$ 0.03	0.04 $\pm$ 0.02	0.01 $\pm$ 0.01
	Aug. Images Only	0.12 $\pm$ 0.03	0.09 $\pm$ 0.01	0.07 $\pm$ 0.05	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Aug. Actions Only	0.01 $\pm$ 0.01	0.01 $\pm$ 0.02	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00
	Fixed Robot Geom.	0.55 $\pm$ 0.12	0.62 $\pm$ 0.09	0.64 $\pm$ 0.07	0.59 $\pm$ 0.13	0.66 $\pm$ 0.05	0.51 $\pm$ 0.08	0.36 $\pm$ 0.05
	MobileVISTA (Ours)	0.65 $\pm$ 0.04	0.62 $\pm$ 0.05	0.58 $\pm$ 0.07	0.59 $\pm$ 0.06	0.59 $\pm$ 0.02	0.49 $\pm$ 0.09	0.41 $\pm$ 0.01

TABLE IV: Detailed ablation results across tasks. Success rate (mean $\pm$ std over three seeds) vs. evaluation radius $r_{eval} \in \{0, 1, 3, 5, 7, 10, 15\}$ cm with yaw $\Delta\theta \sim \mathcal{U}(-5^\circ, 5^\circ)$. **Bold** indicates the best method per column within each task.

truth camera intrinsics and extrinsics from the simulator. This removes a source of error that is orthogonal to the central question of how well each method synthesizes pose-shifted observations.

Second, we assume access to the same robot model used by MobileVISTA when rendering the perturbed scene from the new camera trajectory. This is necessary because, as discussed in Section III-B, a base perturbation induces a

corresponding change in the egocentric camera trajectory through the base-to-camera kinematic chain after action retargeting, and the new camera trajectory cannot be computed without the robot model. Sharing this information with GEN3C ensures both methods use the same target camera trajectory for synthesis.

Finally, GEN3C generates video in fixed 121-frame chunks. We therefore generate each augmented trajectory in

chunks of $121 \times 3 = 363$ frames; trajectories longer than this are truncated at the end. In practice, this truncation affected at most 12 frames per trajectory across the four tasks evaluated, and truncation always occurred at the trailing end of the trajectory.

PSNR Comparison. To quantify reconstruction quality, we render oracle egocentric views from the simulator at perturbed base poses and treat these as ground truth. We then synthesize views at the same perturbed poses using MobileVISTA and GEN3C and compute PSNR against the oracle. For each task, we sample 200 perturbed configurations within a disk of radius $r = 15\text{cm}$ around the canonical pose. Table V reports the mean PSNR across these configurations for each task.

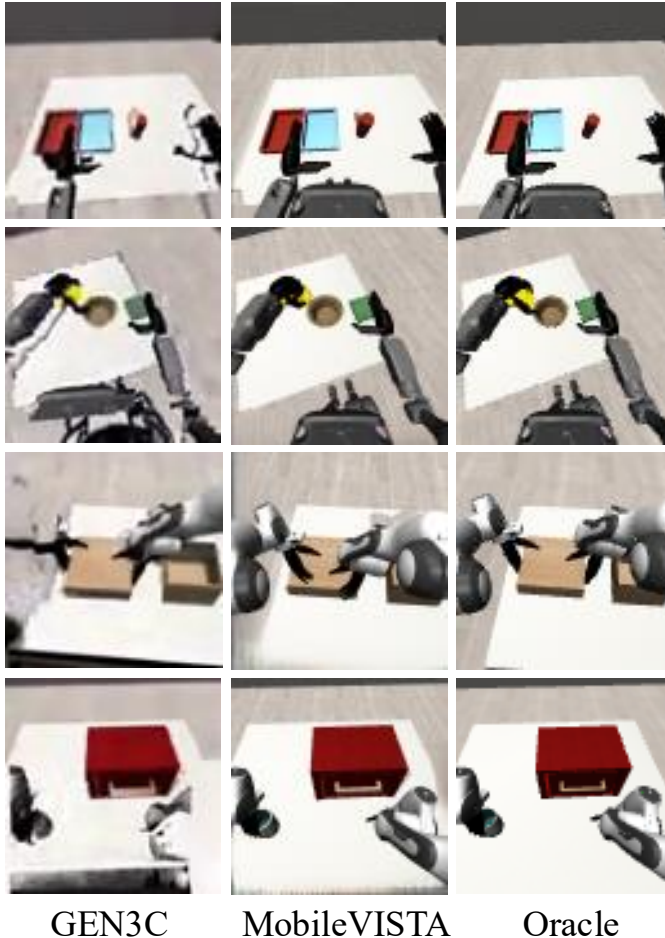


Fig. 8: Qualitative comparison of synthesized egocentric observations. For each task, we show the simulator oracle view, the MobileVISTA-synthesized view, and the GEN3C-synthesized view at the same perturbed base pose. MobileVISTA more faithfully preserves scene geometry and object structure.

MobileVISTA produces higher PSNR than GEN3C across all four tasks, with the largest gaps appearing on the humanoid tasks (Can Sort and Pouring). The trends in Table V are illustrated in the per-trajectory scatter

	GEN3C	M.Vista
Can Sort	14.93	3.81
Pouring	13.65	4.31
Box	14.93	2.39
Drawer	16.85	2.99

TABLE VI: $\downarrow$ Avg. synthesis time per trajectory (minutes). MobileVISTA is 4-7 $\times$ faster than GEN3C across tasks.

visualization in Figure 6 of the main paper.

Qualitative Comparisons. Figure 8 shows representative synthesized observations from MobileVISTA and GEN3C alongside the simulator oracle at matched perturbed base poses. MobileVISTA preserves sharp object boundaries and consistent scene geometry across perturbations, while GEN3C tends to blur object edges and distort surfaces under larger viewpoint shifts. These visual differences are consistent with the PSNR gap reported in Table V.

Data Generation Speed. We measure the wall-clock time required to synthesize one augmented trajectory using each method, averaged across 200 trajectories per task. MobileVISTA timings are measured on a single NVIDIA L40S GPU, and GEN3C timings are measured on a single NVIDIA H200 GPU. Table VI reports per-task averages. MobileVISTA generates augmented trajectories 4-7 $\times$ faster than GEN3C across tasks. Note that as the H200 is a higher-end GPU than the L40S, so the reported speed advantage of MobileVISTA is a conservative estimate of the true gap on matched hardware. We postulate this gap arises as GEN3C performs iterative diffusion sampling for each frame, while MobileVISTA relies on a single forward pass through the geometry-based reprojection pipeline followed by inpainting.

G. Policy Learning from GEN3C-Synthesized Views

We additionally evaluate downstream policy performance when training on GEN3C-synthesized views rather than MobileVISTA-synthesized views. To isolate the effect of the visual synthesis method, we pair GEN3C-synthesized observations with the same action retargeting procedure described in Section III-B, and train diffusion policies using the same configuration as MobileVISTA. We evaluate under the same randomized perturbation regime as Section IV-A.

Results are shown in Figure 5. The Canonical-2D, Simulator (Oracle), and MobileVISTA results are reproduced from Section IV-A for reference. Policies trained on MobileVISTA-synthesized data outperform those trained on GEN3C-synthesized data across all four tasks and across the full range of evaluation perturbation magnitudes. We find the gap in downstream performance is consistent with the reconstruction quality gap reported in Table V: tasks where the two methods produce more similar PSNR also generally show smaller gaps in policy success rate.

H. Hardware Setup

We deploy MobileVISTA and baselines on a Galaxea R1 Pro humanoid robot. The robot is equipped with a head-mounted ZED2 RGB-D camera providing the egocentric view used by all policies, and two wrist-mounted Intel RealSense RGB cameras.

To enable repeatable evaluation across base pose perturbations, we manually mark a grid on the floor extending 15cm in each direction from the canonical robot base pose. Markings are placed by hand and verified against ground truth measurements before evaluation begins. During evaluation, we manually drive the robot to the sampled offset

Method	Can Sort			Pouring			Box			Drawer		
	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm	0-5cm	5-10cm	10-15cm
GEN3C	16.29	14.42	13.08	17.74	15.09	13.28	16.37	13.75	11.78	16.32	15.76	14.35
M.Vista	25.02	24.05	23.28	23.71	23.25	22.18	16.47	16.01	14.72	18.67	18.93	18.29

TABLE V: $\uparrow$ PSNR (dB) between synthesized egocentric views and simulator oracle images across base perturbation magnitudes r .

using these markings as reference, ensuring consistent and repeatable starting configurations across rollouts.

All policy inference is performed on a single NVIDIA RTX 4090 GPU.

I. Data Collection & Real World Task Definitions

Teleoperation. We collect expert demonstrations using the JoyLo teleoperation framework introduced in BEHAVIOR Robot Suite [33]. Demonstrations are recorded in ROS 2 at 10Hz with synchronized RGB-D images, proprioceptive states, and joint-space actions including arm, torso, and gripper commands.

Book Stacking Task Definition. The robot must grasp a vertically placed book from a designated location and reorient it horizontally onto an existing stack inside a bookshelf. We collect 50 expert demonstrations from the canonical (non-perturbed) robot pose, each approximately 150 timesteps in length.

Laundry Sort Task Definition. The robot must sort four laundry items into two bins based on category: two black socks must be placed into a grey bin on the left, and a yellow towel and a beige pair of shorts must be placed into a larger green bin on the right. We do not apply domain randomization to the item placement or appearance. This task is substantially more challenging than Book Stacking due to its longer horizon, the deformability of the items, and the bimanual coordination required to sort multiple objects in sequence.

J. Evaluation Protocol

For each trial, we sample a base translation offset uniformly within a disk of radius $r_{eval} = 15\text{cm}$ centered at the canonical pose. We manually drive the robot to that offset using the floor markings described in Section H. Yaw is not explicitly controlled, since policies are trained over the full $\pm 5^\circ$ yaw range used during augmentation.

From each sampled offset, we roll out all four policies (MobileVISTA, Canonical-2D, DP3, Fine-Tuned $\pi_{0.5}$) without repositioning the robot, enabling direct comparison from identical starting configurations. The order in which policies are evaluated at each offset is randomized and unknown to the experimenter to eliminate bias.

For Book Stacking, we score each rollout in two stages: Grasp, where the robot successfully grasps the book, and Stacked, where the book is successfully placed onto the existing stack. We perform 25 trials at the canonical pose and 25 trials at randomized poses, for a total of 50 trials per policy.

For Laundry Sort, we score each rollout by the number of items correctly sorted into the appropriate bin, ranging from

0 to 4. We perform 25 trials at the canonical pose and 25 trials at randomized poses, for a total of 50 trials per policy.

K. MobileVISTA and Canonical-2D Diffusion Policy

We train MobileVISTA and Canonical-2D policies using Diffusion Policy [29] within the Robomimic framework [35], using the official implementation¹. All hyperparameters are identical across all tasks and all policy variants trained via this pipeline (MobileVISTA, Canonical-2D, Aug. Images Only, Aug. Actions Only, Fixed Robot Geom.). Full hyperparameters are listed in Table VII. For simulated tasks we train for 1400 epochs. For the real-world Book Stacking task we train for 3500 epochs, and for the real-world Laundry task we train for 6500 epochs, where each epoch consists of 500 gradient steps.

The proprioceptive observations differ between simulation and real. In simulation, we use end-effector positions and quaternions for each arm, along with gripper joint positions. In the real-world setting, we instead use joint positions for the left and right arms, torso joint positions, and binary gripper open signals. All three RGB cameras (egocentric and two wrist cameras) are used in both settings.

L. DP3

We train DP3 [30] using the official implementation² with default hyperparameters. Key settings include: observation horizon 2, action horizon 8, prediction horizon 16, DDIM noise scheduler with 100 training steps and 10 inference steps, AdamW optimizer (lr $1\text{e-}4$, weight decay $1\text{e-}6$) with cosine scheduling and 500 warmup steps, batch size 128, and EMA (power 0.75). Point clouds are encoded with a PointNet encoder (output dim 64) using color as an additional input channel regardless of task. For simulated tasks we train for 1400 epochs. Due to compute constraints, real-world policies are trained by global step count rather than epochs: 634,845 steps for Book Stacking and 1.3M steps for Laundry. We train three random seeds per task for all simulated experiments.

M. Fine-Tuned $\pi_{0.5}$

We fine-tune $\pi_{0.5}$ [1] using the official openpi repository³, initializing from the released `pi05_base` checkpoint. We use LoRA fine-tuning (gemma.2b.lora for the vision-language backbone and gemma.300m.lora for the action expert), with EMA disabled. All other training settings follow

¹https://github.com/real-stanford/diffusion_policy

²<https://github.com/YanjieZe/3D-Diffusion-Policy>

³<https://github.com/Physical-Intelligence/openpi>

Hyperparameter	Value
<i>Optimization</i>	
Learning rate	1e-4
L2 regularization	0.0
Batch size	16
Steps per epoch	500
Action normalization	min-max
<i>Policy horizons</i>	
Observation horizon	2
Action horizon	8
Prediction horizon	16
<i>Diffusion</i>	
Noise scheduler	DDIM
Training diffusion steps	100
Inference diffusion steps	10
Beta schedule	squaredcos_cap_v2
Prediction type	ϵ -prediction
EMA power	0.75
<i>UNet</i>	
Channel dims	[256, 512, 1024]
Kernel size	5
Groups	8
Diffusion step embed dim	256
<i>Image encoder</i>	
Backbone	ResNet-18 (no pretraining)
Pooling	SpatialSoftmax (32 keypoints)
Feature dim per camera	64
Input resolution	84×84
Random crop	76×76

TABLE VII: Diffusion Policy hyperparameters used for all MobileVISTA and Canonical-2D policy variants across simulated and real-world tasks.

the `pi05_libero` reference config in the `openpi` repository, with `action_dim=32` and `action_horizon=16`.

Simulated experiments. For each task, we train three random seeds for up to 30,000 steps, saving a checkpoint every 1,000 steps. For each seed, we sweep over all saved checkpoints by evaluating 15 rollouts at each of three perturbation radii ($r \in \{0, 5, 10\}$ cm) and selecting the checkpoint with the highest mean success rate across all three radii. We then run full evaluation (50 rollouts per radius, all perturbation magnitudes) on the selected checkpoint for each seed, and report results averaged over the three seeds.

Real-world experiments. For the real-world Book Stacking task, we average the selected checkpoint steps across the three best-performing simulated seeds to obtain a single fine-tuning step count of 22,000 steps. For the real-world Laundry task, we scale this by the ratio of Robomimic diffusion policy training durations between the two real-world tasks ($6500/3500 \approx 1.86\times$), yielding 41,000 steps. This scaling aligns the relative fine-tuning budget of $\pi_{0.5}$ with that of the Robomimic-based diffusion policies across the two tasks.